DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GEN AI AND ITS APPLICATIONS

UNIT 4 HANDS-ON

NAME : CHINMAYI SHRAVANI YELLANKI
SRN : PES2UG23CS152
CLASS AND SEC : SEM 6 - C
DATE : 23.04.2026

NOTEBOOK 1 : MULTIMODAL MODELS

PART 1 : CLIP

1.1

PART 1.1

```
# Load a sample image (we'll use a publicly available image)
image_url = "http://images.cocodataset.org/val2017/000000039769.jpg"  # two cats on a couch
image = load_image(image_url)

# Display the image
image
```

1.2

```python
    probs = outputs.logits_per_image.softmax(dim=1).squeeze().tolist()
    return dict(zip(labels, probs))


# Test on our cat image
labels = ["cat", "dog", "bird", "car", "airplane", "sofa", "person"]
results = clip_zero_shot_classify(image, labels, clip_model, clip_processor, device)

print("Zero-Shot Classification Results:")
print("-" * 40)
for label, prob in sorted(results.items(), key=lambda x: -x[1]):
    bar = "" * int(prob * 40)
    print(f"{label:<10} {prob:.1%}  {bar}")
```

```
Zero-Shot Classification Results:
----------------------------------------
cat        79.8%  ################################
sofa       18.9%  #######
person     0.6%
dog        0.4%
car        0.2%
airplane   0.1%
bird       0.1%
```

```python
# Try with a different image - let's test a dog image
dog_image_url = "https://cdn.shopify.com/s/files/1/0086/0795/7054/files/Golden-Retriever.jpg?v=1645179525"
dog_image = load_image(dog_image_url)

results_dog = clip_zero_shot_classify(dog_image, labels, clip_model, clip_processor, device)

print("Zero-Shot Classification - Dog image:")
print("-" * 40)
for label, prob in sorted(results_dog.items(), key=lambda x: -x[1]):
    bar = "" * int(prob * 40)
    print(f"{label:<10} {prob:.1%}  {bar}")
```

```
Zero-Shot Classification - Dog image:
----------------------------------------
dog        99.1%  ########################################
person     0.4%
sofa       0.3%
cat        0.2%
bird       0.1%
airplane   0.0%
car        0.0%
```

1.3

```python
def get_clip_image_embedding(image, model, processor, device):
    inputs = processor(images=image, return_tensors="pt").to(device)
    with torch.no_grad():
        embedding_output = model.get_image_features(**inputs)
        embedding = embedding_output.pooler_output if hasattr(embedding_output, 'pooler_output') else embedding_output
    return F.normalize(embedding, dim=-1)

def get_clip_text_embedding(text, model, processor, device):
    inputs = processor(text=[text], return_tensors="pt", padding=True).to(device)
    with torch.no_grad():
        embedding_output = model.get_text_features(**inputs)
        embedding = embedding_output.pooler_output if hasattr(embedding_output, 'pooler_output') else embedding_output
    return F.normalize(embedding, dim=-1)

# Compare: cat image vs. text descriptions
cat_emb = get_clip_image_embedding(image, clip_model, clip_processor, device)

comparisons = [
    "two cats sleeping on a sofa",
    "a dog running in a park",
    "feline animals resting indoors",  # paraphrase — tests semantic understanding
    "a vehicle on a road"
]

print("Cosine similarity: cat image vs. text descriptions")
print("-" * 55)
for text in comparisons:
    text_emb = get_clip_text_embedding(text, clip_model, clip_processor, device)
    sim = (cat_emb @ text_emb.T).item()
    bar = "" * int((sim + 1) * 20)  # normalize to positive range
    print(f"{sim:.4f}  {bar}")
    print(f"        '{text}'")
```

```
Cosine similarity: cat image vs. text descriptions
-------------------------------------------------------
0.2995  ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
        'two cats sleeping on a sofa'
0.1127  ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
        'a dog running in a park'
0.2821  ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
        'feline animals resting indoors'
0.1553  ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
        'a vehicle on a road'
```

PART 2 : BLIP

2.1

```python
# Test on multiple images to see the range of BLIP's captioning
test_images = {
    "cats on couch": image,
    "labrador dog": dog_image,
}

# Add a third image — a landscape
landscape_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/1/1a/24701-nature-natural-beauty.jpg/1280px-24701-nature-natural-beauty.jpg"
try:
    landscape_image = load_image(landscape_url)
    test_images["landscape"] = landscape_image
except:
    print("Could not load landscape image, continuing with 2 images.")

print("BLIP Captions:")
print("-" * 50)
for name, img in test_images.items():
    caption = blip_caption(img, blip_caption_model, blip_caption_processor, device)
    print(f"[{name}]")
    print(f"  Caption: {caption}")
    print()
```

```
BLIP Captions:
--------------------------------------------------
[cats on couch]
  Caption: two cats sleeping on a couch

[labrador dog]
  Caption: a dog sitting in a basket with a green background

[landscape]
  Caption: a field with a tree and the sun setting
```

2.2

```
VQA — Cats on couch image:
--------------------------------------------------
Q: How many cats are in the image?
A: 2

Q: What are the cats doing?
A: sleeping

Q: What color are the cats?
A: brown

Q: Where are the cats sitting?
A: couch

Q: Is it daytime or nighttime?
A: daytime
```

```python
# VQA on the dog image
dog_questions = [
    "What breed of dog is this?",
    "What color is the dog?",
    "Is the dog indoors or outdoors?"
]

print("VQA — Labrador image:")
print("-" * 50)
for question in dog_questions:
    answer = blip_vqa(dog_image, question, blip_vqa_model, blip_vqa_processor, device)
    print(f"Q: {question}")
    print(f"A: {answer}")
    print()
```

```
VQA — Labrador image:
--------------------------------------------------
Q: What breed of dog is this?
A: lab

Q: What color is the dog?
A: tan

Q: Is the dog indoors or outdoors?
A: outdoors
```

PART 3 : WHISPER

3.1

```python
    """
    # Whisper requires 16kHz audio
    if sampling_rate != 16000:
        import librosa
        audio_array = librosa.resample(audio_array, orig_sr=sampling_rate, target_sr=16000)
        sampling_rate = 16000

    # Extract log-mel spectrogram features
    input_features = processor(
        audio_array,
        sampling_rate=sampling_rate,
        return_tensors="pt"
    ).input_features.to(device)

    # Generate transcription
    forced_decoder_ids = processor.get_decoder_prompt_ids(
        language="english", task=task
    )

    with torch.no_grad():
        predicted_ids = model.generate(
            input_features,
            forced_decoder_ids=forced_decoder_ids
        )

    return processor.batch_decode(predicted_ids, skip_special_tokens=True)[0]


# Transcribe
transcript = whisper_transcribe(audio_array, sampling_rate, whisper_model, whisper_processor, device)

print("Whisper Transcription Result:")
print("-" * 50)
print(f"Whisper output:  '{transcript}'")
print(f"Ground truth:    '{sample['text']}'")
```

```
Using custom `forced_decoder_ids` from the (generation) config. This is deprecated in favor of the `task` and `language` flags/config options.
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention_mask` to obtain reliable results.
A custom logits processor of type <class 'transformers.generation.logits_process.SuppressTokensLogitsProcessor'> has been passed to `.generate()`, but it was also created in `.generate()`, given its parameterization. The custom <class 'transforme
A custom logits processor of type <class 'transformers.generation.logits_process.SuppressTokensAtBeginLogitsProcessor'> has been passed to `.generate()`, but it was also created in `.generate()`, given its parameterization. The custom <class 'tra
Whisper Transcription Result:
--------------------------------------------------
Whisper output:  ' Mr. Quilter is the apostle of the middle classes, and we are glad to welcome his gospel.'
Ground truth:    'MISTER QUILTER IS THE APOSTLE OF THE MIDDLE CLASSES AND WE ARE GLAD TO WELCOME HIS GOSPEL'
```

3.2

```
[27]
2s
    lang_to_id = model.generation_config.lang_to_id

    language_codes = list(lang_to_id.keys())
    language_token_ids = list(lang_to_id.values())

    # Extract language logits
    lang_logits = logits[0, 0, language_token_ids]
    lang_probs = torch.softmax(lang_logits, dim=-1)

    # Top 5
    top5_probs, top5_indices = torch.topk(lang_probs, 5)

    top5_langs = [language_codes[i] for i in top5_indices.tolist()]
    top5_scores = top5_probs.tolist()

    return list(zip(top5_langs, top5_scores))


# Run
lang_results = whisper_detect_language(
    audio_array,
    sampling_rate,
    whisper_model,
    whisper_processor,
    device
)

# Pretty print
print("Language Detection — Top 5:")
print("-" * 40)
for lang_token, prob in lang_results:
    lang_name = lang_token.replace("<|", "").replace("|>", "")
    bar = "" * int(prob * 50)
    print(f"{lang_name:<12} {prob:.2%}  {bar}")
```

```
Language Detection — Top 5:
----------------------------------------
en           99.92%
la           0.05%
nn           0.01%
cy           0.00%
ja           0.00%
```

3.3

```python
# Transcribe the first 3 samples from the dataset
print("Batch transcription — 3 samples:")
print("=" * 60)

for i, sample in enumerate(ds.select(range(3))):
    audio, sr = load_audio_sample(sample)

    transcript = whisper_transcribe(audio, sr, whisper_model, whisper_processor, device)
    reference = sample["text"]

    print(f"Sample {i+1}:")
    print(f"  Whisper:    {transcript}")
    print(f"  Reference:  {reference}")
    # Simple word accuracy
    w_words = set(transcript.lower().split())
    r_words = set(reference.lower().split())
    overlap = len(w_words & r_words) / max(len(r_words), 1)
    print(f"  Word overlap: {overlap:.0%}")
    print()
```

```
Batch transcription — 3 samples:
============================================================
Sample 1:
  Whisper:    Mr. Quilter is the apostle of the middle classes, and we are glad to welcome his gospel.
  Reference:  MISTER QUILTER IS THE APOSTLE OF THE MIDDLE CLASSES AND WE ARE GLAD TO WELCOME HIS GOSPEL
  Word overlap: 81%

Sample 2:
  Whisper:    Nor is Mr. Quilter's manner less interesting than his matter.
  Reference:  NOR IS MISTER QUILTER'S MANNER LESS INTERESTING THAN HIS MATTER
  Word overlap: 80%

Sample 3:
  Whisper:    He tells us that at this festive season of the year, with Christmas and roast beef looming before us, similarly is drawn from eating and its results occur most readily to the mind.
  Reference:  HE TELLS US THAT AT THIS FESTIVE SEASON OF THE YEAR WITH CHRISTMAS AND ROAST BEEF LOOMING BEFORE US SIMILES DRAWN FROM EATING AND ITS RESULTS OCCUR MOST READILY TO THE MIND
  Word overlap: 90%
```

PART 4

```python
# Now simulate the FULL pipeline: audio → text → image search → caption
# We use the LibriSpeech audio and connect it to our image search

# Step 1: Whisper transcribes the audio
spoken_query_audio, spoken_query_sr = load_audio_sample(ds[0])

print("Step 1 — Whisper: Transcribing audio...")
spoken_text = whisper_transcribe(spoken_query_audio, spoken_query_sr,
                                 whisper_model, whisper_processor, device)
print(f"  Transcribed: '{spoken_text}'")

# Step 2: CLIP finds the most relevant image
print("\nStep 2 — CLIP: Finding most relevant image...")
results = multimodal_search(
    spoken_text, db_embeddings, image_db,
    clip_model, clip_processor,
    blip_caption_model, blip_caption_processor,
    device, top_k=1
)

# Step 3: BLIP captions the result
print(f"\nStep 3 — BLIP: Caption of top result [{results[0]['name']}]:")
print(f"  {results[0]['caption']}")
print(f"  (similarity score: {results[0]['similarity']:.4f})")

print("\n" + "=" * 60)
print("Pipeline complete:")
print(f"  Audio → '{spoken_text}'")
print(f"  Best image match: {results[0]['name']}")
print(f"  Caption: {results[0]['caption']}")
```

```
Step 1 — Whisper: Transcribing audio...
  Transcribed: ' Mr. Quilter is the apostle of the middle classes, and we are glad to welcome his gospel.'

Step 2 — CLIP: Finding most relevant image...

Step 3 — BLIP: Caption of top result [red_car]:
  a woman walking down the street
  (similarity score: 0.2122)

============================================================
Pipeline complete:
  Audio → ' Mr. Quilter is the apostle of the middle classes, and we are glad to welcome his gospel.'
  Best image match: red_car
  Caption: a woman walking down the street
```

NOTEBOOK 2 : AGENTIC WORKFLOWS

```python
print("\n" + "="*80)
print("📊 RESEARCH OUTPUT\n")
print(research_output)

print("\n" + "="*80)
print("📄 ARTICLE\n")
print(article)

print("\n" + "="*80)
print("🌍 TRANSLATION (SPANISH)\n")
print(translated)
```

```
...
================================================================================
📊 RESEARCH OUTPUT

CONTINUE

**Additional Insights:**

1. **Ethical Considerations:** As LLMs become more advanced, there is a growing need to address ethical concerns, such as bias, transparency, and accountability (Source: Ethics in AI: A Review of the Current State of the Field).
2. **Explainability and Interpretability:** Researchers are working to improve the explainability and interpretability of LLMs, enabling better understanding of their decision-making processes (Source: Explainable AI: A Review of the Current Stat
3. **Multimodal Learning:** LLMs are being extended to multimodal learning, enabling them to process and generate multiple forms of data, such as text, images, and audio (Source: Multimodal Learning: A Review of the Current State of the Field).
4. **Real-World Applications:** LLMs are being applied to various real-world problems, including healthcare, finance, and education, with promising results (Source: Large Language Models in Healthcare: A Review of the Current State of the Field).

**Future Directions:**

1. **Continued Advancements in LLM Training:** Researchers will continue to explore new techniques and architectures for training LLMs, leading to further improvements in their performance and capabilities.
2. **Increased Adoption in Industry:** LLMs will become more widely adopted in various industries, leading to increased efficiency, productivity, and innovation.
3. **Addressing Ethical and Societal Concerns:** As LLMs become more pervasive, there will be a growing need to address ethical and societal concerns, such as job displacement, bias, and transparency.

**Conclusion:**
The field of large language models is rapidly evolving, with significant advancements in recent years. As LLMs continue to improve, we can expect to see increased adoption, new applications, and further innovations in the field.

================================================================================
📄 ARTICLE

Since there is no new information to process, I will say .

================================================================================
🌍 TRANSLATION (SPANISH)

Dado que no hay un artículo proporcionado para traducir, no puedo continuar con la traducción.
```